

Semantic Structure & Special Characters

Landmark tags & character entities — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep **VS Code** and your **browser** open side by side. For every task, **attempt and write your answer *before* we discuss it.**

Part 1 — The Semantic Skeleton

TRY THIS · VS CODE

In the last session, you learned about **semantic vs. non-semantic tags** and used text-formatting elements like `<strong>`, `<em>` and `<ins>`. Reopen the `Full_Stack_Sessions` folder in **Visual Studio Code**, create a new folder `3_HTML`, and inside it a new file named `index.html`. Write the boilerplate code using `!`, then copy-paste some old HTML content (headings, paragraphs, and the text-formatting tags you used last session) into the body so we have something to restructure.

Note: Pull the latest changes from GitHub before you start, in case anything changed on the remote.

```
git pull
```

Q1. Right now, is your content wrapped in plain `<div>` tags, or do you even have any wrapper around it?

Q2. Before we learn anything new — if you had to guess, what part of a webpage would you call its **"header"**? What do you usually find there on real websites?

Q3. What about the **"footer"**? Where do you expect to find it on a webpage?

Q4. Most websites also have a strip of links near the top (Home, About, Contact, ...). What would you call that part?

Concept — Landmark semantic tags. In class 07 you learned that semantic tags describe their meaning to the browser and the developer, unlike generic `<div>` containers. HTML gives us a set of "**landmark**" semantic tags specifically meant for page layout:

- `<header>` — introductory content, usually a logo/title and navigation
- `<nav>` — a block of navigation links
- `<main>` — the primary, unique content of the page (only one per page)
- `<section>` — a thematic grouping of content, usually with its own heading
- `<article>` — a self-contained piece of content that could stand on its own (a blog post, a news story, a product card)
- `<aside>` — content indirectly related to the main content (a sidebar, a related-links box, an ad)
- `<footer>` — closing content, usually copyright, contact info, or links

Q5. Compare your guesses in Q2–Q4 with the definitions above. How close were you?

TRY THIS · RESTRUCTURE WITH LANDMARK TAGS

Restructure your `index.html` using these tags: wrap your top title and a small `nav` (with 2–3 fake links) in `<header>` and `<nav>`; wrap your main content in `<main>`; put your old paragraphs inside one or more `<article>` or `<section>` tags (whichever fits the content); add a small sidebar-like block using `<aside>` (e.g. "Related links" or "About the author"); and wrap your closing info in `<footer>`. Example skeleton:

```
<body>
  <header>
    <h1>My Page</h1>
    <nav>
      <a href="#">Home</a>
      <a href="#">About</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>My First Article</h2>
      <p>...</p>
    </article>
    <aside>
      <p>Related links...</p>
    </aside>
  </main>

  <footer>
    <p>...</p>
  </footer>
</body>
```

Q6. Does the page look any different **visually** after this change? What actually changed then?

Q7. Can a page have **more than one** `<header>` or `<footer>`? Try adding a small `<header>` and `<footer>` inside your `<article>` itself (e.g. article title on top, "posted by ..." at the bottom) and see if the browser complains.

Q8. Can you have **more than one** `<main>` tag on the same page?

Q13. You learned in class 07 that extra spaces typed in your code get collapsed into one. Try adding ` ` three or four times in a row inside a paragraph instead of pressing spacebar. What is the difference?

TRY THIS · COPYRIGHT LINE

Add a copyright line to your `<footer>`, using `©` and the current year, e.g.:

```
<footer>
  <p>&copy; 2026 My Page. All rights reserved.</p>
</footer>
```

Q14. Add your page/company name next to the copyright line and mark it as a trademark using `™`.

Q15. Somewhere on your page, write the sentence **"Terms & Conditions"** — but type it using the `&` entity instead of typing `&` directly. What happens if you just type `&` on its own?

Q16. Try showing the literal text `<p>` somewhere on your page (as visible text, not as an actual tag) using `<` and `>`. Where might a real website need to do this? (*Hint: think about tutorial or documentation websites.*)

Q17. Do a quick search for **"HTML entity list"**. Pick two more entities you find interesting and use them somewhere on your page. Write down what they are and what they show.

CLASS WRAP-UP · GIT

Push all this code to GitHub and verify if everything works on the live link as well.

```
git add .
git commit -m "Semantic structure and character entities"
git push
```